WEEK 3 · LESSON 16 OF 24 · HANDOUT

Privacy and Data Protection

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=NUk6QN02UxQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon US 2020: Privacy-preserving methods — Building secure projects — https://www.youtube.com/watch?v=jKlJEPSZAYo

🎓 **Go deeper:** PyCon US 2022: Open Source, Python Based Tools For Data Privacy — https://www.youtube.com/watch?v=tuhJiz9_V6o

🎓 **Go deeper:** PyCon US 2023: The ChatGPT Privacy Tango — Data Security and Large Language Models — https://www.youtube.com/watch?v=sGn6BuldvIk

Your goal in this lesson: Apply Rwanda's data-protection principles to an ML system with data minimisation, pseudonymisation, k-anonymity checks, differential privacy for statistics, and deletion and retention rules.

The law and the principles

Rwanda's **Law No. 058/2021 relating to the protection of personal data and privacy**, supervised by the National Cyber Security Authority (NCSA), sets rules for anyone who processes personal data. Key ideas you must design for:

- **Lawful basis and purpose** — collect data for a clear, stated purpose (for example consent or a contract).
- **Data minimisation** — collect and keep only what you need.
- **Accuracy and storage limits** — keep data correct and only as long as needed.
- **Security** — protect data against loss and misuse.
- **Rights of the data subject** — people can ask to see, correct or delete their data, and to object.
- **Storage outside Rwanda** — allowed only under conditions in the law.

This lesson is not legal advice: check the current law and NCSA guidance for your project. Many other countries have similar laws (for example the EU's GDPR).

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

1. Minimise

```
raw = applicants.assign(
    full_name=[f"Applicant {i}" for i in applicants["applicant_id"]],
    phone=["07" + str(8_800_000_00 + i)[-8:] for i in applicants["applicant_id"]],
)
MODEL_NEEDS = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month", "repaid"]
AUDIT_NEEDS = ["age", "gender", "area"]                    # kept apart, for fairness checks
only
print("collected:", list(raw.columns))
print("model table:", MODEL_NEEDS)
print("dropped for modelling:", [c for c in raw.columns if c not in MODEL_NEEDS + AUDIT_NEEDS +
["applicant_id"]])
```

The model never needs names or phone numbers, so they never enter the modelling data.

2. Pseudonymise

Replace IDs with a **keyed hash** (HMAC). Without the secret key, nobody can reverse it or recompute it — but you can still join tables:

```
import hashlib
import hmac
import os

KEY = os.environ.get("PSEUDONYM_KEY", "demo-key-use-a-real-secret").encode()

def pseudonym(value):
    return hmac.new(KEY, str(value).encode(), hashlib.sha256).hexdigest()[:16]

model_table = raw[MODEL_NEEDS].assign(pid=raw["applicant_id"].map(pseudonym))
audit_table = raw[AUDIT_NEEDS].assign(pid=raw["applicant_id"].map(pseudonym))
print(model_table.head(3).to_string(index=False))
print("plain SHA-256 of ID 1 (anyone can recompute it!):", hashlib.sha256(b"1").hexdigest()
[:16])
```

A plain hash of an ID is **not** private: someone can hash all possible IDs and match them. The secret key prevents that. Pseudonymised data is still personal data under the law — but much safer.

3. Check re-identification risk with k-anonymity

Even without names, a rare combination (a 62-year-old rural man) can identify someone. **k-anonymity** asks: is every combination of "quasi-identifiers" shared by at least k people?

```
def smallest_group(df, columns):
    return int(df.groupby(columns).size().min())

print("exact age + gender + area: smallest group =", smallest_group(audit_table, ["age",
"gender", "area"]))
banded = audit_table.assign(age_band=pd.cut(audit_table["age"], [0, 30, 45, 120], labels=["
<30", "30-45", "45+"]))
print("age band + gender + area:  smallest group =", smallest_group(banded, ["age_band",
"gender", "area"]))
```

With exact ages, some combinations have only a handful of people — risky to share. With age **bands**, every group has many people. Generalise data before sharing it for research or reports.

4. Differential privacy for published statistics

When you publish counts ("how many rural borrowers defaulted"), adding carefully sized random noise protects each individual: the result barely changes whether any one person is in the data or not.

```
rng = np.random.default_rng(0)

def private_count(true_count, epsilon):
    return true_count + rng.laplace(0, 1 / epsilon)          # one person changes a count by at
most 1

defaults = applicants.groupby("area")["repaid"].apply(lambda s: int((s == 0).sum()))
for eps in [0.1, 1.0]:
    noisy = {a: round(private_count(c, eps)) for a, c in defaults.items()}
    print(f"epsilon {eps}: true {defaults.to_dict()} -> published {noisy}")
```

Smaller **epsilon** = more noise = more privacy. For large counts the noise hardly matters; for small groups it protects people strongly.

5. Deletion and retention

```
def delete_person(applicant_id, *tables):
    p = pseudonym(applicant_id)
    return [t[t["pid"] != p] for t in tables]

model_table, audit_table = delete_person(17, model_table, audit_table)
print("rows left:", len(model_table), len(audit_table))

RETENTION = {"raw application forms": "5 years after the loan closes (check your legal
obligations)",
             "model training tables": "until the next retraining, then replaced",
             "prediction logs": "12 months, pseudonymised"}
for item, rule in RETENTION.items():
    print(f"- {item}: {rule}")
```

Remember: deleting a person from the data does not remove them from an **already trained** model. Plan regular retraining so deletions take effect, and record when each request was handled.

Key terms

- **Personal data** — information about an identifiable person.
- **Data minimisation** — collecting and keeping only what is needed.
- **Pseudonymisation** — replacing identifiers with codes that need a secret to link back.
- **k-anonymity** — every combination of quasi-identifiers is shared by at least k people.
- **Differential privacy** — adding calibrated noise so results barely depend on any one person.
- **Retention policy** — how long each kind of data is kept.

Remember this: Design privacy in: collect less, separate audit data, pseudonymise with a secret key, check re-identification risk, add noise to published statistics, and honour deletion and retention rules under Rwanda's data-protection law.

Name: _____ Date: _____

Activity

Task 1

Find the age-band boundaries that give a smallest group of at least 100 people.

Task 2

Publish the number of defaulters per area and gender with epsilon 0.5. Which counts change most, relatively?

Task 3

Write a one-paragraph privacy notice an applicant could understand.

Task 4

List every place personal data appears in your Week 1 service (logs, backups, test files…).

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Which Rwandan law protects personal data and privacy?
   - A. The Tax Code
   - B. The Road Traffic Law
   - C. Law No. 058/2021
   - D. There is none
2. What is data minimisation?
   - A. Deleting the model
   - B. Compressing files
   - C. Collecting and keeping only the data you need
   - D. Using small fonts
3. Why is a plain SHA-256 hash of an ID not private?
   - A. Anyone can hash all possible IDs and match them
   - B. It cannot be stored
   - C. It is random
   - D. It is too long
4. What does k-anonymity check?
   - A. Password length
   - B. Model accuracy
   - C. Encryption strength
   - D. That every combination of quasi-identifiers is shared by at least k people
5. In differential privacy, what does a smaller epsilon mean?
   - A. Faster code
   - B. No effect
   - C. Less noise
   - D. More noise and more privacy
6. Does deleting a person's data remove them from an already trained model?
   - A. No — retraining is needed for the deletion to take effect
   - B. Models never contain data
   - C. Yes, instantly
   - D. Only on Fridays

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.